Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 4

По дисциплине «Современные платформы программирования»

Выполнил:
Абрамчук В. Л.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
03.04.2026

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Ход работы:

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

1. Автоматический анализ трендов популярных репозиториев Условие:

Напишите Python-скрипт, который анализирует тренды популярных GitHub-репозиториев за определённый период (например, за последние 7 или 30 дней).

1. Запрашивает у пользователя:

- Язык программирования (например, Python, JavaScript, Go)
- Период анализа (за неделю, за месяц)
- (Опционально) Минимальное количество звезд, чтобы фильтровать менее популярные проекты

2. Использует GitHub API для получения самых быстрорастущих репозиториев по звёздам (stars).

3. Для каждого найденного репозитория собирает данные:

- Название
- Автор
- Количество звёзд за период
- Общее количество звёзд
- Количество форков
- Основной язык
- Описание
- Ссылка на репозиторий

4. Определяет самые динамично растущие проекты (по количеству новых звёзд за период).

5. Визуализирует динамику роста популярных проектов (matplotlib, seaborn).

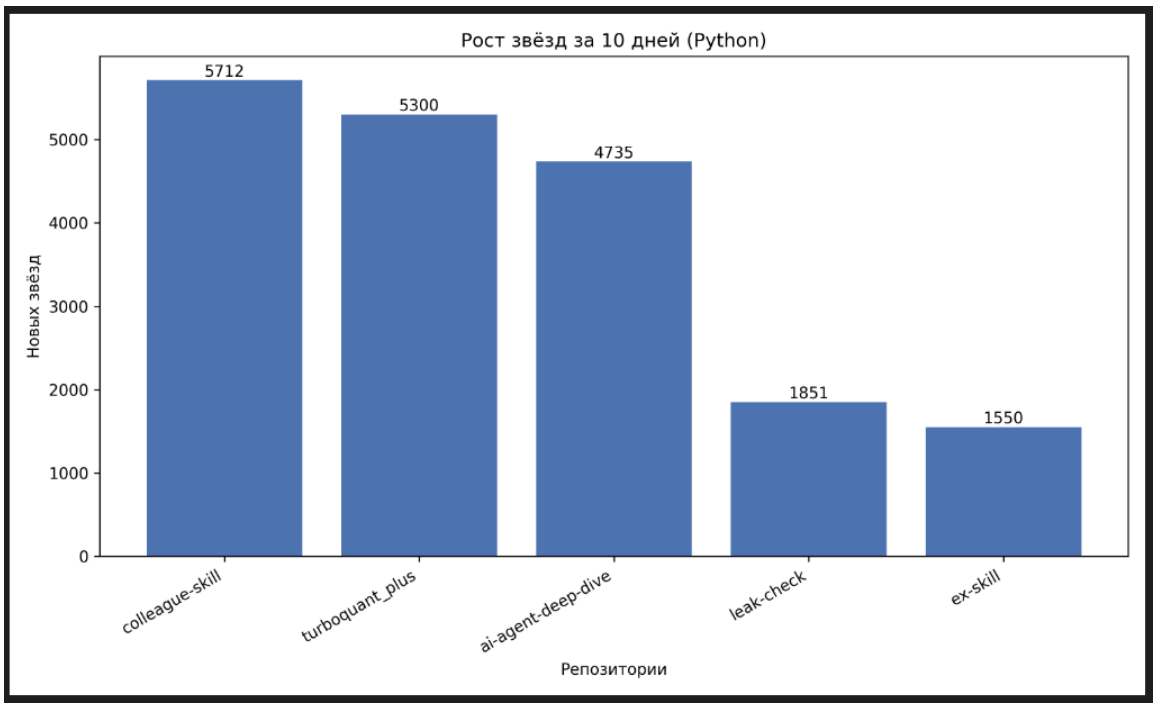
```
from datetime import datetime, timedelta, timezone
from time import sleep

import requests
import matplotlib.pyplot as plt

GITHUB_API = "https://api.github.com"
SEARCH_API = f"{GITHUB_API}/search/repositories"
GITHUB_TOKEN = ""

HEADERS = {
    "Accept": "application/vnd.github.v3.star+json",
    "Authorization": f"token {GITHUB_TOKEN}"
}

# Ввод
def get_user_input():
    lang = input("Введите язык программирования: ").strip()
    try:
        days = int(input("Период анализа (7 / 30): ").strip())
    except ValueError:
        days = 7

    min_stars_input = input("Минимум звёзд (опционально): ").strip()

    if min_stars_input.isdigit():
        min_stars = int(min_stars_input)
    else:
        min_stars = None

    return lang, days, min_stars

# Поиск репозиториев
def search_repositories(lang, days, min_stars):
    start_date = (datetime.now() - timedelta(days=days)).strftime('%Y-%m-%d')
    query = f"language:{lang} created:>{start_date}"

    if min_stars:
        query += f" stars:>={min_stars}"

    params = {
        "q": query,
        "sort": "stars",
        "order": "desc",
        "per_page": 10
    }

    r = requests.get(SEARCH_API, params=params, headers={"Accept":
"application/vnd.github.v3+json"}, timeout=5)
    r.raise_for_status()

    result = r.json().get("items", [])

    return result

# Подсчёт новых звёзд
def count_new_stars(owner, repo, days):
    url = f"{GITHUB_API}/repos/{owner}/{repo}/stargazers"
    cutoff_date = datetime.now(timezone.utc) - timedelta(days=days)
    page = 1
    new_stars = 0

    while True:
        params = {"per_page": 100, "page": page}
        r = requests.get(url, headers=HEADERS, params=params, timeout=5)

        if r.status_code != 200:
            break

        stars = r.json()
        if not stars:
            break

        for star in stars:
```

```
            starred_at = datetime.fromisoformat(star["starred_at"].replace("Z", "+00:00"))
            if starred_at >= cutoff_date:
                new_stars += 1
            else:
                return new_stars

        page += 1
        sleep(0.2)

    return new_stars

# Обработка
def process_repositories(items, days):
    repos = []

    for repo in items:
        name = repo["name"]
        owner = repo["owner"]["login"]

        print(f"Подсчёт новых звёзд для {owner}/{name} ...")
        new_stars = count_new_stars(owner, name, days)

        repos.append({
            "name": name,
            "author": owner,
            "language": repo["language"],
            "desc": repo["description"] or "Без описания",
            "url": repo["html_url"],
            "forks": repo["forks_count"],
            "total_stars": repo["stargazers_count"],
            "new_stars": new_stars
        })

    repos.sort(key=lambda x: x["new_stars"], reverse=True)
    return repos[:5]

# Вывод
def print_repositories(repos):
    print("\nТОП быстрорастущих репозиториев:\n")
    for i, r in enumerate(repos, 1):
        print(
            f"{i}. {r['name']} ({r['language']})\n"
            f"   Автор: {r['author']}\n"
            f"   Новых ☆ за период: {r['new_stars']}\n"
            f"   Всего ☆: {r['total_stars']:,}\n"
            f"   Форков: {r['forks']}\n"
            f"   {r['desc']}\n"
            f"   {r['url']}\n"
        )

# График
def plot_growth(repos, lang, days):
    names = [r["name"] for r in repos]
    growth = [r["new_stars"] for r in repos]

    plt.figure(figsize=(10, 6))

    bars = plt.bar(names, growth, color="#4C72B0")

    for rect in bars:
        y = rect.get_height()
        plt.text(rect.get_x() + rect.get_width()/2, y, f"{y}", ha="center", va="bottom")

    filename = f"trending_growth_{lang}.png"

    plt.title(f"Рост звёзд за {days} дней ({lang})")
    plt.xlabel("Репозитории")
    plt.ylabel("Новых звёзд")
    plt.xticks(rotation=30, ha="right")
    plt.tight_layout()
    plt.savefig(filename, dpi=300)

    print(f'График сохранён в "{filename}"')
```

```python
def main():
    lang, days, min_stars = get_user_input()
    items = search_repositories(lang, days, min_stars)

    if not items:
        print("Репозитории не найдены.")
        return

    repos = process_repositories(items, days)

    print_repositories(repos)
    plot_growth(repos, lang, days)

if __name__ == "__main__":
    main()
```

```
Введите язык программирования: Python
Период анализа (7 / 30): 10
Минимум звёзд (опционально): 500
Подсчёт новых звёзд для titanwings/colleague-skill ...
Подсчёт новых звёзд для TheTom/turboquant_plus ...
Подсчёт новых звёзд для tvytlx/ai-agent-deep-dive ...
Подсчёт новых звёзд для garinasset/leak-check ...
Подсчёт новых звёзд для therealXiaomanChu/ex-skill ...
Подсчёт новых звёзд для oaker-io/wewrite ...
Подсчёт новых звёзд для jxnxts/mcp-brasil ...
Подсчёт новых звёзд для HKUDS/OpenHarness ...
Подсчёт новых звёзд для wquguru/harness-books ...
Подсчёт новых звёзд для adamlyttleapps/claude-skill-aso-appstore-screenshots ...
ТОП быстрорастущих репозиториев:

1. colleague-skill (Python)
   Автор: titanwings
   Новых ★ за период: 5712
   Всего ★: 5,711
   Форков: 341
   将冰冷的离别化为温暖的 Skill, 欢迎加入数字生命1.0! Transforming cold farewells into warm skills? It's giving rebirth era. Welcome to Digital Life 1.0. 
   https://github.com/titanwings/colleague-skill

2. turboquant_plus (Python)
   Автор: TheTom
   Новых ★ за период: 5300
   Всего ★: 5,300
   Форков: 717
   Без описания
   https://github.com/TheTom/turboquant_plus

3. ai-agent-deep-dive (Python)
   Автор: tvytlx
   Новых ★ за период: 4735
   Всего ★: 4,730
   Форков: 1456
   AI Agent 源码深度研究报告
   https://github.com/tvytlx/ai-agent-deep-dive

4. leak-check (Python)
   Автор: garinasset
   Новых ★ за период: 1851
   Всего ★: 1,851
   Форков: 235
   个人信息 “泄漏” 检测接口
   https://github.com/garinasset/leak-check

5. ex-skill (Python)
   Автор: therealXiaomanChu
   Новых ★ за период: 1550
   Всего ★: 1,549
   Форков: 162
   把前任蒸馏成 AI Skill, 用ta的方式跟你说话。Inspired by colleague-skill (同事skill) .
   https://github.com/therealXiaomanChu/ex-skill
```

```
График сохранён в "trending_growth_Python.png"
```



Вывод: научился работать с Github API, приобрёл практические навыки написания программ для работы с REST API или GraphQL API